Can go to unsafe from a safe state

e.g. at time t_1, T_2 is given another resource!

Avoidance Algos

a- Resource Allocation Graphs

Useful if one resource instance per resource type.

Cycle detection algo $O(n^2)$

DIY.

b- Banker's Algo

- Multiple instances ✓
- Less efficient than RAG.
- Data Structures n Processes
  m Resource Types

i. Available of length m.

ii. Max: A matrix of $n \times m$.
if $M_{ij} = k \Rightarrow T_i$ needs max k instances of resources of type R_j

iii. Allocation: A matrix of $n \times m$.
if $M_{ij} = k \Rightarrow T_i$ given k instances of type R_j

iv. Need: A matrix of $n \times m$.
$Need_{ij} = Max_{ij} - Allocation_{ij}$

Two Algos: a- Safety Algo b- Resource-Request Algo.

a- Safety Algo

whether a given state is safe or

1- Work a vector of length m.
Finish " " " " n.

Work = Available.
Finish = false.

2- Find i s.t.
Finish$_i$ = false and.
Need$_i$ ≤ Work.
If no such i, Goto 4

$O(m \times n^2)$

3- Work += Allocation$_i$;
Finish$_i$ = true.
Goto 2.

4- If Finish$_i$ = true for all i
then Safe state.

---

5. Resource Request Algo

Request$_i$ be the request vector for T_i.
Request$_{ij}$ = k ⇒ T_i requests k instance of R_j

1- If Request$_i$ ≤ Need$_i$, Goto 2 else ERROR.
2- If Request$_i$ ≤ Available, Goto 3 else WAIT.
3- Pretend to fulfill the request and Run the Safety Algo.

Available -= Request$_i$;
Allocation$_i$ += Request$_i$;
Need$_i$ -= Request$_i$;

6 Five Threads

Three Resources A B C

10 5 7

Given that

	Allocation	Max	Available
T_0	0 1 0	7 5 3	3 3 2
T_1	2 0 0	3 2 2	
T_2	3 0 2	9 0 2	
T_3	2 1 1	2 2 2	
T_4	0 0 2	4 3 3	

	Need
T_0	7 4 3
T_1	1 2 2
T_2	6 0 0
T_3	0 1 1
T_4	4 3 1

Safe or not

Work = {3 3 2}

Finish = {F, F, F, F, F}

~~i ≠ Finish$_i$ == f and Work ≥ Need$_i$~~

i	Finish$_i$ == f	Need$_i$ <= Work	New Work	New Finish Seq
0	✓	{7 4 3} ≤ {3 3 2}	No need	
1	✓	{1 2 2} ≤ {3 3 2}	{5, 3, 2}	{F T F F F} <T_1>
0	✓	7 4 3 ≤ 5 3 2		
2	✓	6 0 0 ≤ 5 3 2	need resources	
3	✓	0 1 1 ≤ 5 3 2	7 4 3	F T F T F T_1 T_3
0	✓	7 4 3 ≤ 7 4 3 ✓	7 5 3	T T F T F T_1 T_3 T_0
2	✓	6 0 0 ≤ 7 5 3 ✓	10 9 5	T T T T F T_1 T_3 T_0 T_2
4	✓	4 3 1 ≤ 10 9 5 ✓	10 5 7 same	T T T T T T_1 T_3 T_0 T_2 T_4

Work = {3, 3, 2} Available
Finish = {F, F, F, F, F}

i	Finish$_i$ == F	Need$_i$ ≤ Work	Work + Alloc.	Finish	Seq
0	~~T~~	743 ≤ 332 (F)			
1	T	122 ≤ 332 (T)	{5, 3, 2}	FTFFF	T_1
2	T	600 ≤ 532 (F)			
3	T	011 ≤ 532 (T)	{7 4 3}	FTFTF	T_1 T_3
4	T	431 ≤ 743 (T)	{7 4 5}	FTFTT	T_1 T_3 T_4
0	T	743 ≤ 745 (T)	{7 5 5}	TTFTT	T_1 T_3 T_4 T_0
2	T	600 ≤ 755 (T)	{10 5 7}	TTTTT	T_1 T_3 T_4 T_0 T_2

Resource Request

T_1 → Request$_1$ = (1, 0, 2)

(1, 0, 2) ≤ Need$_1$ (1, 2, 2) ✓

(1, 0, 2) ≤ Available (3, 3, 2) ✓

Pretend Available (2, 3, 0) {Available -= Request$_1$}

	Allocation	Max	Available	Need
	0 1 0	7 5 3	2 3 0	7 4 3
Request$_1$ →	+ (3 0 2)	3 2 2		≥ 0 2 0
	3 0 2	9 0 2		6 0 0
	2 1 1	2 2 2		0 1 1
	0 0 2	4 3 3		4 3 1

Initial Work = (2, 3, 0) Available
Finish = (F, F, F, F, F)

	Finish$_i$ == F	Need$_i$ ≤ Work	Work + Alloc$_i$	Finish	Seq
1	✓	(0 2 0) ≤ (2 3 0) ✓	(5 3 2)	F T F F F	T_1
2	✓	6 0 0 ≤ 5 3 2 ✗			
3	✓	0 1 1 ≤ 5 3 2 ✓	(7 4 3)	F T F T F	T_1 T_3
4	✓	4 3 1 ≤ ~~7~~ 4 3 ✓	(7 4 5)	F T F T T	T_1 T_3 T_4
0	✓	7 4 3 ≤ 7 4 5 ✓	(7 5 5)	T T F T T	T_1 T_3 T_4 T_0
2	✓	6 0 0 ≤ 7 5 5 ✓	(10 5 7)	T T T T T	T_1 T_3 T_4 T_0 T_2

Safe Seq = (T_1 T_3 T_4 T_0 T_2) ✓

~~What if~~ Next T_4 Requests. (3, 3, 1) ✓

Request$_4$ = (3, 3, 0) ≤ Need$_4$

Request$_4$ ≤ Available

(3, 3, 0) ≤ (2, 3, 0) → False → ~~ERROR~~ WAIT ✓

Next T_0 Requests.

Request$_0$ = (0, 2, 0) ≤ Need$_0$

(0, 2, 0) ≤ (7, 4, 3) → ✓

Request$_0$ ≤ Available ?

(0, 2, 0) ≤ (2, 3, 0) ✓

Pretend Now, ~~Run the Safety Algo~~ ~~Pretend~~ ~~Initial Work = Available~~

New Available −= Request$_0$

Available = (2, 1, 0) ✓

Allocation$_0$ += Request$_0$ (0 3 0)

Need$_0$ −= Request$_0$ (7 2 3)

Available = (2, 1, 0)

	Allocation	Max	Need	Available
P0	0 3 0	7 5 3	7 2 3	(2 1 0)
P1	3 0 2	3 2 2	0 2 0	
P2	3 0 2	9 0 2	6 0 0	
P3	2 1 1	2 2 2	0 1 1	
P4	0 0 2	4 3 3	4 3 1	

Work = (2, 1, 0)

Finish = (F, F, F, F, F)

i	Finish$_i$ == F	Need$_i$ ≤ Work	Work += Alloc$_i$	Finish	Seq
0	✓	723 ≤ 210 X			
1	✓	020 ≤ 210 X			
2	✓	600 ≤ 210 X			
3	✓	011 ≤ 210 X			
4	✓	431 ≤ 210 X			

Wat
a Safe State
Revert Back.

---

c- Deadlock Detection

- An algo to detect
- " " " recover.

Single instance A lot of overhead.

Build a Wait-For Graph for the RAG.

$O(n^2)$

Multiple Instances

Similar to banker's algo.

→ Available of leyth m.

→ Allocation $n \times m$.

→ Request $n \times m$

$Request_{ij} = k \Rightarrow$

T_i needs k instances of R_j

Step 1 Work = Available

$$Finish \longrightarrow \begin{cases} Finish_i = false, & Allocation_i \neq 0 \\ Finish_i = True, & Allocation_i = 0 \end{cases}$$

Step 2 Find an index i s.t. both:

a- $Finish_i$ = False and

b- $Request_i \leq Work$.

If no such i, goto Step 4.

Step 3

Work += Allocation$_i$

Finish$_i$ = true

Goto Step 2.

Step 4 If for some i, Finish$_i$ = ~~true~~ false.

then T_i is deadlocked.

$O(m \times n^2)$

Example T_0 T_1 T_2 T_3 T_4

Resources R_0 R_1 R_2.

7 2 6

	Allocation	Request	Available
T_0	0 1 0	0 0 0	0 0 0
T_1	2 0 0	2 0 2	
T_2	3 0 3	0 0 0	
T_3	2 1 1	1 0 0	
T_4	0 0 2.	0 0 2.	

Initially, Work = (0, 0, 0)

Finish = (F, F, F, F, F)

i	Finish$_i$ == F	Request$_i$ ≤ Work	New Work	New Finish	Seq
0	✓	(000) ≤ (0,00) ✓	→ (010)	TFFFF	T_0
1	✓	(202) ≤ (010) X			
2	✓	(000) ≤ (010) ✓	(313)	TFTFF	T_0 T_2.
3	✓	(100) ≤ (313) ✓	(524)	TFTTF	T_0 T_2 T_3.
4	✓	(002) ≤ (524) ✓	(526)	TFTTT	T_0 T_2 T_3 T_4.
1	✓	(202) ≤ (526) ✓	(536)	TTTTT	T_0 T_2 T_3 T_4 T_1
		202	(726)		

No deadlock

Homework Test fn.

Request

000

202 ?

001

100.

002

	Allocation	Request	Available
	010	000	000
T_1	200	202	
T_2	303	001	
T_3	211	100	
T_4	002	002	

Initial, Work = Available = (0, 0, 0)
Finish = (F F F F F)

i	Finish$_i$ = F	Request$_i$ ≤ Work	New Work	New Finish	Seq
0	✓	000 ≤ 000 ✓	010	TFFFF	T_0
1	✓	202 ≤ 010 ✗			
2	✓	001 ≤ 010 ✗			
3	✓	100 ≤ 010 ✗			
4	✓	002 ≤ 010 ✗			

Deadlocked States $\langle T_1, T_2, T_3, T_4 \rangle$.

Detection Algo Usage

When to invoke?

- How often deadlocks are likely to occur
- How many threads are affected.

→ At defined intervals or when CPU utilization drops below a certain level
→ Whenever a request cannot be granted immediately. vs.
→ If algo is run at arbitrary intervals then we won't know which thread's request initiated the deadlock.

Recovery from Deadlocks

→ Manually by operator.

→ Automatic

a- Abort Process / Thread

→ All at once or

→ One by one.

Try not to abort interactive.

b- Resource Preemption

Take resource from one thread and give to another.

Issues:

- Victim selection
- Rollback to a consistant state or a complete rollback.

c- Avoid Starvation: If the same "victim" is selected repeatedly.